Chapter 3. Technical Infrastructure

Free software projects rely on technologies that support the selective capture and integration of information. The more skilled you are at using these technologies, and at persuading others to use them, the more successful your project will be. This only becomes more true as the project grows. Good information management is what prevents open source projects from collapsing under the weight of Brooks' Law,[1] which states that adding manpower to a late software project makes it later. Fred Brooks observed that the complexity of a project increases as the *square* of the number of participants. When only a few people are involved, everyone can easily talk to everyone else, but when hundreds of people are involved, it is no longer possible for each person to remain constantly aware of what everyone else is doing. If good free software project management is about making everyone feel like they're all working together in the same room, the obvious question is: what happens when everyone in a crowded room tries to talk at once?

This problem is not new. In non-metaphorical crowded rooms, the solution is *parliamentary procedure*: formal guidelines for how to have real-time discussions in large groups, how to make sure important dissents are not lost in floods of "me-too" comments, how to form subcommittees, how to recognize when decisions are made, etc. An important part of parliamentary procedure is specifying how the group interacts with its information management system. Some remarks are made "for the record", others are not. The record itself is subject to direct manipulation, and is understood to be not a literal transcript of what occurred, but a representation of what the group is willing to *agree* occurred. The record is not monolithic, but takes different forms for different purposes. It comprises the minutes of individual meetings, the complete collection of all minutes of all meetings, summaries, agendas and their annotations, committee reports, reports from correspondents not present, lists of action items, etc.

Because the Internet is not really a room, we don't have to worry about replicating those parts of parliamentary procedure that keep some people quiet while others are speaking. But when it comes to information management techniques, well-run open source projects are parliamentary procedure on steroids. Since almost all communication in open source projects happens in writing, elaborate systems have evolved for routing and labeling data appropriately; for minimizing repetitions so as to avoid spurious divergences; for storing and retrieving data; for correcting bad or obsolete information; and for associating disparate bits of information with each other as new connections are observed. Active participants in open source projects internalize many of these techniques, and will often perform complex manual tasks to ensure that information is routed correctly. But the whole endeavor ultimately depends on sophisticated software support. As much as possible, the communications media themselves should do the routing, labeling, and recording, and should make the information available to humans in the most convenient way possible. In practice, of course, humans will still need to intervene at many points in the process, and it's important that the software make such interventions convenient too. But in general, if the humans take care to label and route information accurately on its first entry into the system, then the software should be configured to make as much use of that metadata as possible.

The advice in this chapter is intensely practical, based on experiences with specific software and usage patterns. But the point is not just to teach a particular collection of techniques. It is also to demonstrate, by means of many small examples, the overall attitude that will best encourage good information management in your project. This attitude will involve a combination of technical skills and people skills. The technical skills are essential because information management software always requires configuration, plus a certain amount of ongoing maintenance and tweaking as new needs arise (for example, see the discussion of how to handle project growth in the section called “Pre-Filtering the Bug Tracker” later in this chapter). The people skills are necessary because the human community also requires maintenance: it's not always immediately obvious how to use these tools to full advantage, and in some cases projects have conflicting conventions (for example, see the discussion of setting `Reply-to` headers on outgoing mailing list posts, in the section called “Mailing Lists”). Everyone involved with the project will need to be encouraged, at the right times and in the right ways, to do their part to keep the project's information well organized. The more involved the contributor, the more complex and specialized the

[1] From his book *The Mythical Man Month*, 1975. See http://en.wikipedia.org/wiki/The_Mythical_Man-Month and http://en.wikipedia.org/wiki/Brooks_Law.

techniques she can be expected to learn.

Information management has no cut-and-dried solution. There are too many variables. You may finally get everything configured just the way you want it, and have most of the community participating, but then project growth will make some of those practices unscalable. Or project growth may stabilize, and the developer and user communities settle into a comfortable relationship with the technical infrastructure, but then someone will come along and invent a whole new information management service, and pretty soon newcomers will be asking why your project doesn't use it—for example, this is happening now to a lot of free software projects that predate the invention of the wiki (see http://en.wikipedia.org/wiki/Wiki). Many questions are matters of judgement, involving tradeoffs between the convenience of those producing information and the convenience of those consuming it, or between the time required to configure information management software and the benefit it brings to the project.

Beware of the temptation to over-automate, that is, to automate things that really require human attention. Technical infrastructure is important, but what makes a free software project work is care—and intelligent expression of that care—by the humans involved. The technical infrastructure is mainly about giving humans convenient ways to do that.

What a Project Needs

Most open source projects offer at least a minimum, standard set of tools for managing information:

Web site
: Primarily a centralized, one-way conduit of information from the project out to the public. The web site may also serve as an administrative interface for other project tools.

Mailing lists
: Usually the most active communications forum in the project, and the "medium of record."

Version control
: Enables developers to manage code changes conveniently, including reverting and "change porting". Enables everyone to watch what's happening to the code.

Bug tracking
: Enables developers to keep track of what they're working on, coordinate with each other, and plan releases. Enables everyone to query the status of bugs and record information (e.g., reproduction recipes) about particular bugs. Can be used for tracking not only bugs, but also tasks, releases, new features, etc.

Real-time chat
: A place for quick, lightweight discussions and question/answer exchanges. Not always archived completely.

Each tool in this set addresses a distinct need, but their functions are also interrelated, and the tools must be made to work together. Below we will examine how they can do so, and more importantly, how to get people to use them. The web site is not discussed until the end, since it acts more as glue for the other components than as a tool unto itself.

You may be able to avoid a lot of the headache of choosing and configuring these tools by using a *canned hosting* site: a server that offers prepackaged, templatized web areas with all the accompanying tools needed to run a free software project. See the section called “Canned Hosting” later in this chapter for a discussion of the advantages and disadvantages of canned hosting.

Mailing Lists

Mailing lists are the bread and butter of project communications. If a user is exposed to any forum besides the web pages, it is most likely to be one of the project's mailing lists. But before they experience the mailing list itself, they will experience the mailing list interface—that is, the mechanism by which they join ("subscribe to") the list. This brings us to Rule #1 of mailing lists:

> *Don't try to manage mailing lists by hand—get list management software.*

It will be tempting to put this off. Setting up mailing list management software might seem like overkill at first. Managing small, low-traffic lists by hand will seem seductively easy: you just set up a subscription address that forwards to you, and when someone mails it, you add (or remove) their email address in some text file that holds all the addresses on the list. What could be simpler?

The trick is that good mailing list management—which is what people have come to expect—is not simple at all. It's not just about subscribing and unsubscribing users when they request. It's also about moderating to prevent spam, offering the mailing list in digest versus message-by-message form, providing standard list and project information by means of auto-responders, and various other things. A human being monitoring a subscription address can supply only a bare minimum of functionality, and even then not as reliably and promptly as software could.

Modern list management software usually offers at least the following features:

Both email- and web-based subscription
: When a user subscribes to a list, she should *promptly* get an automated welcome message in reply, telling her what she has subscribed to, how to interact further with the mailing list software, and (most importantly) how to unsubscribe. This automatic reply can be customized to contain project-specific information, of course, such as the project's web site, FAQ location, etc.

Subscription in either digest mode or message-by-message mode
: In digest mode, the subscriber receives one email per day, containing all the list activity for that day. For people who are following a list loosely, without participating, digest mode is often preferable, because it allows them to scan all the subjects at once and avoid the distraction of emails coming in at random times.

Moderation features
: To "moderate" is to check posts to make sure they are a) not spam, and b) on topic, before they go out to the entire list. Moderation necessarily involves humans, but software can do a lot to make it easier. There is more said about moderation later.

Administrative interface
: Among other things, this enables an administrator to go in and remove obsolete addresses easily. This can become urgent when a recipient's address starts sending automatic "I am no longer at this address" replies back to the list in response to every list post. (Some mailing list software can even detect this by itself and unsubscribe the person automatically.)

Header manipulation
: Many people have sophisticated filtering and replying rules set up in their mail readers. Mailing list software can add and manipulate certain standard headers for these people to take advantage of (more details below).

Archiving
: All posts to the managed lists are stored and made available on the web; alternatively, some mailing list software offers special interfaces for plugging in an external archiving tool such as MHonArc (http://www.mhonarc.org/). As the section called “Conspicuous Use of Archives” in Chapter 6, *Communications* discusses, archiving is crucial.

The point of all this is merely to emphasize that mailing list management is a complex problem that has

been given a lot of thought, and mostly been solved. You certainly don't need to become an expert in it. But you should be aware that there's always room to learn more, and that list management will occupy your attention from time to time in the course of running a free software project. Below we'll examine a few of the most common mailing list configuration issues.

Spam Prevention

Between when this sentence is written and when it is published, the Internet-wide spam problem will probably double in severity—or at least it will feel that way. There was a time, not so long ago, when one could run a mailing list without taking any spam-prevention measures at all. The occasional stray post would still show up, but infrequently enough to be only a low-level annoyance. That era is gone forever. Today, a mailing list that takes no spam prevention measures will quickly be submerged in junk emails, to the point of unusability. Spam prevention is mandatory.

We divide spam prevention into two categories: preventing spam posts from appearing on your mailing lists, and preventing your mailing list from being a source of new email addresses for spammers' harvesters. The former is more important, so we examine it first.

Filtering posts

There are three basic techniques for preventing spam posts, and most mailing list software offers all three. They are best used in tandem:

1. **Only auto-allow postings from list subscribers.**

   This is effective as far as it goes, and also involves very little administrative overhead, since it's usually just a matter of changing a setting in the mailing list software's configuration. But note that posts which aren't automatically approved must not be simply discarded. Instead, they should be passed along for moderation, for two reasons. First, you want to allow non-subscribers to post. A person with a question or suggestion should not need to subscribe to a mailing list just to make a single post there. Second, even subscribers may sometimes post from an address other than the one by which they're subscribed. Email addresses are not a reliable method of identifying people, and shouldn't be treated as such.

2. **Filter posts through spam-filtering software.**

   If the mailing list software makes it possible (most do), you can have posts filtered by spam-filtering software. Automatic spam-filtering is not perfect, and never will be, since there is a never-ending arms race between spammers and filter writers. However, it can greatly reduce the amount of spam that gets through to the moderation queue, and since the longer that queue is the more time humans must spend examining it, any amount of automated filtering is beneficial.

   There is not space here for detailed instructions on setting up spam filters. You will have to consult your mailing list software's documentation for that (see the section called “Software” later in this chapter). List software often comes with some built-in spam prevention features, but you may want to add some third-party filters. I've had good experiences with these two: SpamAssassin (http://spamassassin.apache.org/) and SpamProbe (http://spamprobe.sourceforge.net/). This is not a comment on the many other open source spam filters out there, some of which are apparently also quite good. I just happen to have used those two myself and been satisfied with them.

3. **Moderation.**

   For mails that aren't automatically allowed by virtue of being from a list subscriber, and which make it through the spam filtering software, if any, the last stage is *moderation*: the mail is routed to a special address, where a human examines it and confirms or rejects it.

Confirming a post takes one of two forms: you can accept the post just this once, or you can tell the list software to allow this and all future posts from the same sender. You almost always want to do the latter, in order to reduce the future moderation burden. Details on how to confirm vary from system to system, but it's usually a matter of replying to a special address with the command "accept" (meaning accept just this one post) or "allow" (allow this and future posts).

Rejecting is usually done by simply ignoring the moderation mail. If the list software never receives confirmation that something is a valid post, then it won't pass that post on to the list, so simply dropping the moderation mail achieves the desired effect. Sometimes you also have the option of responding with a "reject" or "deny" command, to automatically disapprove future mails from the same sender without even running them through moderation. There is rarely any point doing this, since moderation is mostly about spam prevention, and spammers tend not to send from the same address twice anyway.

Be sure to use moderation *only* for filtering out spams and clearly off-topic messages, such as when someone accidentally posts to the wrong mailing list. The moderation system will usually give you a way to respond directly to the sender, but don't use that method to answer questions that really belong on the mailing list itself, even if you know the answer off the top of your head. To do so would deprive the project's community of an accurate picture of what sorts of questions people are asking, and deprive them of a chance to answer questions themselves and/or see answers from others. Mailing list moderation is strictly about keeping the list free of junk and off-topic emails, nothing more.

Address hiding in archives

To prevent your mailing lists from being a source of addresses for spammers, a common technique is for the archives to obscure people's email addresses, for example by replacing

```
jrandom@somedomain.com
```

with

```
jrandom_AT_somedomain.com
```

or

```
jrandomNOSPAM@somedomain.com
```

or some similarly obvious (to a human) encoding. Since spam address harvesters often work by crawling through web pages—including your mailing list's online archives—and looking for sequences containing "@", encoding the addresses is a way of making people's email addresses invisible or useless to spammers. This does nothing to prevent spam from being sent to the mailing list itself, of course, but it does avoid increasing the amount of spam sent directly to list users' personal addresses.

Address hiding can be controversial. Some people like it a lot, and will be surprised if your archives don't do it automatically. Other people think it's too much of an inconvenience (because humans also have to translate the addresses back before using them). Sometimes people assert that it's ineffective, because a harvester could in theory compensate for any consistent encoding pattern. However, note that there is empirical evidence that address hiding *is* effective, see http://www.cdt.org/speech/spam/030319spamreport.shtml.

Ideally, the list management software would leave the choice up to each individual subscriber, either through a special yes/no header or a setting in that subscriber's list account preferences. However, I don't know of any software which offers per-subscriber or per-post choice in the matter, so for now the list

manager must make a decision for everyone (assuming the archiver offers the feature at all, which is not always the case). I lean very mildly toward turning address hiding on. Some people are very careful to avoid posting their email addresses on web pages or anywhere else a spam harvester might see it, and they would be disappointed to have all that care thrown away by a mailing list archive; meanwhile, the inconvenience address hiding imposes on archive users is very slight, since it's trivial to transform an obscured address back to a valid one if you need to reach the person. But keep in mind that, in the end, it's still an arms race: by the time you read this, harvesters might well have evolved to the point where they can recognize most common forms of hiding, and we'll have to think of something else.

Identification and Header Management

List subscribers often want to put mails from the list into a project-specific folder, separate from their other mail. Their mail reading software can do this automatically by examining the mail's *headers*. The headers are the fields at the top of the mail that indicate the sender, recipient, subject, date, and various other things about the message. Certain headers are well known and effectively mandatory:

```
From: ...
To: ...
Subject: ...
Date: ...
```

Others are optional, though still quite standard. For example, emails are not strictly required to have the

```
Reply-to: sender@email.address.here
```

header, but most do, because it gives recipients a foolproof way to reach the author (it is especially useful when the author had to send from an address other than the one to which replies should be directed).

Some mail reading software offers an easy-to-use interface for filing mails based on patterns in the Subject header. This leads people to request that the mailing list add an automatic prefix to all Subjects, so they can set their readers to look for that prefix and automatically file the mails in the right folder. The idea is that the original author would write:

```
Subject: Making the 2.5 release.
```

but the mail would show up on the list looking like this:

```
Subject: [discuss@lists.example.org] Making the 2.5 release.
```

Although most list management software offers the option to do this, I strongly recommend against turning the option on. The problem it solves can easily be solved in much less obtrusive ways, and the cost of eating space in the Subject field is far too high. Experienced mailing list users typically scan the Subjects of the day's incoming list mail to decide what to read and/or respond to. Prepending the list's name to the Subject can push the right side of the Subject off the screen, rendering it invisible. This obscures information that people depend on to decide what mails to open, thus reducing the overall functionality of the mailing list for everyone.

Instead of munging the Subject header, teach your users to take advantage of the other standard headers, starting with the To header, which should say the mailing list's name:

```
To: <discuss@lists.example.org>
```

Any mail reader that can filter on Subject should be able to filter on To just as easily.

There are a few other optional-but-standard headers expected for mailing lists. Filtering on these is even more reliable than using the "To" or "Cc" headers; since these headers are added to each post by the mailing list management software itself, some users may be counting on their presence:

```
list-help: <mailto:discuss-help@lists.example.org>
list-unsubscribe: <mailto:discuss-unsubscribe@lists.example.org>
list-post: <mailto:discuss@lists.example.org>
Delivered-To: mailing list discuss@lists.example.org
Mailing-List: contact discuss-help@lists.example.org; run by ezmlm
```

For the most part, they are self-explanatory. See http://www.nisto.com/listspec/list-manager-intro.html for more explanation, or if you need the really detailed, formal specification, see http://www.faqs.org/rfcs/rfc2369.html.

Notice how these headers imply that if you have a mailing list named "list", then you also have administrative addresses "list-help" and "list-unsubscribe" available. In addition to these, it is normal to have "list-subscribe", for joining, and "list-owner", for reaching the list administrators. Depending on the list management software you use, these and/or various other administrative addresses may be set up; the documentation will have details. Usually a complete explanation of all these special addresses is mailed to each new user as part of an automated "welcome mail" on subscribing. You yourself will probably get a copy of this welcome mail. If you don't, then ask someone else for a copy, so you know what your users are seeing when they join the list. Keep the copy handy so you can answer questions about the mailing list functions, or better yet, put it on a web page somewhere. That way when people lose their own copy of the instructions and post to ask "How do I unsubscribe from this list?", you can just hand them the URL.

Some mailing list software offers an option to append unsubscription instructions to the bottom of every post. If that option is available, turn it on. It causes only a couple of extra lines per message, in a harmless location, and it can save you a lot of time, by cutting down on the number of people who mail you—or worse, mail the list!—asking how to unsubscribe.

The Great Reply-to Debate

Earlier, in the section called “Avoid Private Discussions”, I stressed the importance of making sure discussions stay in public forums, and talked about how active measures are sometimes needed to prevent conversations from trailing off into private email threads; furthermore, this chapter is all about setting up project communications software to do as much of the work for you as possible. Therefore, if the mailing list management software offers a way to automatically cause discussions to stay on the list, you would think turning that feature on would be the obvious choice.

Well, not quite. There is such a feature, but it has some pretty severe disadvantages. The question of whether or not to use it is one of the hottest debates in mailing list management—admittedly, not a controversy that's likely to make the evening news in your city, but it can flare up from time to time in free software projects. Below, I will describe the feature, give the major arguments on both sides, and make the best recommendation I can.

The feature itself is very simple: the mailing list software can, if you wish, automatically set the Reply-to header on every post to redirect replies to the mailing list. That is, no matter what the original sender puts in the Reply-to header (or even if they don't include one at all), by the time the list subscribers see the post, the header will contain the list address:

```
Reply-to: discuss@lists.example.org
```

On its face, this seems like a good thing. Because virtually all mail reading software pays attention to the Reply-to header, now when anyone responds to a post, their response will be automatically addressed to the entire list, not just to the sender of the message being responded to. Of course, the responder can still manually change where the message goes, but the important thing is that *by default* replies are directed to the list. It's a perfect example of using technology to encourage collaboration.

Unfortunately, there are some disadvantages. The first is known as the *Can't Find My Way Back Home* problem: sometimes the original sender will put their "real" email address in the Reply-to field, because for one reason or another they send email from a different address than where they receive it. People who always read and send from the same location don't have this problem, and may be surprised that it even exists. But for those who have unusual email configurations, or who cannot control how the From address on their mails looks (perhaps because they send from work and do not have any influence over the IT department), using Reply-to may be the only way they have to ensure that responses reach them. When such a person posts to a mailing list that he's not subscribed to, his setting of Reply-to becomes essential information. If the list software overwrites it, he may never see the responses to his post.

The second disadvantage has to do with expectations, and in my opinion is the most powerful argument against Reply-to munging. Most experienced mail users are accustomed to two basic methods of replying: *reply-to-all* and *reply-to-author*. All modern mail reading software has separate keys for these two actions. Users know that to reply to everyone (that is, including the list), they should choose reply-to-all, and to reply privately to the author, they should choose reply-to-author. Although you want to encourage people to reply to the list whenever possible, there are certainly circumstances where a private reply is the responder's prerogative—for example, they may want to say something confidential to the author of the original message, something that would be inappropriate on the public list.

Now consider what happens when the list has overridden the original sender's Reply-to. The responder hits the reply-to-author key, expecting to send a private message back to the original author. Because that's the expected behavior, he may not bother to look carefully at the recipient address in the new message. He composes his private, confidential message, one which perhaps says embarrassing things about someone on the list, and hits the send key. Unexpectedly, a few minutes later his message appears *on the mailing list!* True, in theory he should have looked carefully at the recipient field, and should not have assumed anything about the Reply-to header. But authors almost always set Reply-to to their own personal address (or rather, their mail software sets it for them), and many longtime email users have come to expect that. In fact, when a person deliberately sets Reply-to to some other address, such as the list, he usually makes a point of mentioning this in the body of the message, so people won't be surprised at what happens when they reply.

Because of the possibly severe consequences of this unexpected behavior, my own preference is to configure list management software to never touch the Reply-to header. This is one instance where using technology to encourage collaboration has, it seems to me, potentially dangerous side-effects. However, there are also some powerful arguments on the other side of this debate. Whichever way you choose, you will occasionally get people posting to your list asking why you didn't choose the other way. Since this is not something you ever want as the main topic of discussion on your list, it might be good to have a canned response ready, of the sort that's more likely to stop discussion than encourage it. Make sure you do *not* insist that your decision, whichever it is, is obviously the only right and sensible one (even if you think that's the case). Instead, point out that this is a very old debate, there are good arguments on both sides, no choice is going to satisfy all users, and therefore you just made the best decision you could. Politely ask that the subject not be revisited unless someone has something genuinely new to say, then stay out of the thread and hope it dies a natural death.

Someone may suggest a vote to choose one way or the other. You can do that if you want, but I personally do not feel that counting heads is a satisfactory solution in this case. The penalty for someone who is surprised by the behavior is so huge (accidentally sending a private mail to a public list), and the inconvenience for everyone else is fairly slight (occasionally having to remind someone to respond to the whole list instead of just to you), that it's not clear that the majority, even though they are the majority,

should be able to put the minority at such risk.

I have not addressed all aspects of this issue here, just the ones that seemed of overriding importance. For a full discussion, see these two canonical documents, which are the ones people always cite when they're having this debate:

- **Leave Reply-to alone**, *by Chip Rosenthal*

  http://www.unicom.com/pw/reply-to-harmful.html

- **Set Reply-to to list**, *by Simon Hill*

  http://www.metasystema.net/essays/reply-to.mhtml

Despite the mild preference indicated above, I do not feel there is a "right" answer to this question, and happily participate in many lists that *do* set Reply-to. The most important thing you can do is settle on one way or the other early, and try not to get entangled in debates about it after that.

Two fantasies

Someday, someone will get the bright idea to implement a *reply-to-list* key in a mail reader. It would use some of the custom list headers mentioned earlier to figure out the address of the mailing list, and then address the reply directly to the list only, leaving off any other recipient addresses, since most are probably subscribed to the list anyway. Eventually, other mail readers will pick up the feature, and this whole debate will go away. (Actually, the Mutt [http://www.mutt.org/] mail reader does offer this feature.[2])

An even better solution would be for Reply-to munging to be a per-subscriber preference. Those who want the list to set Reply-to munged (either on others' posts or on their own posts) could ask for that, and those who don't would ask for Reply-to to be left alone. However, I don't know of any list management software that offers this on a per-subscriber basis. For now, we seem to be stuck with a global setting.[3]

Archiving

The technical details of setting up mailing list archiving are specific to the software that's running the list, and are beyond the scope of this book. When choosing or configuring an archiver, consider these qualities:

Prompt updating
: People will often want to refer to an archived post made within the last hour or two. If possible, the archiver should archive each post instantaneously, so that by the time a post appears on the mailing list, it's already present in the archives. If that option isn't available, then at least try to set the archiver to update itself every hour or so. (By default, some archivers run their update processes once per night, but in practice that's far too much lag time for an active mailing list.)

Referential stability
: Once a message is archived at a particular URL, it should remain accessible at that exact same URL forever, or as close to forever as possible. Even if the archives are rebuilt, restored from backup, or otherwise fixed, any URLs that have already been made publicly available should remain the same. Stable references make it possible for Internet search engines to index the archives, which is a major boon to users looking for answers. Stable references are also important because mailing list posts

[2] Shortly after this book appeared, Michael Bernstein [http://www.michaelbernstein.com/] wrote me to say: "There are other email clients that implement a reply-to-list function besides Mutt. For example, Evolution has this function as a keyboard shortcut, but not a button (Ctrl+L)."

[3] Since I wrote that, I've learned that there is at least one list management system that offers this feature: Siesta [http://siesta.unixbeard.net/]. See also this article about it: http://www.perl.com/pub/a/2004/02/05/siesta.html

and threads are often linked to from the bug tracker (see the section called “Bug Tracker”) later in this chapter or from other project documents.

Ideally, mailing list software would include a message's archive URL, or at least the message-specific portion of the URL, in a header when it distributes the message to recipients. That way people who have a copy of the message would be able to know its archive location without having to actually visit the archives, which would be helpful because any operation that involves one's web browser is automatically time-consuming. Whether any mailing list software actually offers this feature, I don't know; unfortunately, the ones I have used do not. However, it's something to look for (or, if you write mailing list software, it's a feature to consider implementing, please).

Backups
: It should be reasonably obvious how to back up the archives, and the restoration recipe should not be too difficult. In other words, don't treat your archiver as a black box. You (or someone in your project) should know where it's storing the messages, and how to regenerate the actual archive pages from the message store if it should ever become necessary. Those archives are precious data—a project that loses them loses a good part of its collective memory.

Thread support
: It should be possible to go from any individual message to the *thread* (group of related messages) that that original message is part of. Each thread should have its own URL too, separate from the URLs of the individual messages in the thread.

Searchability
: An archiver that doesn't support searching—on the bodies of messages, as well as on authors and subjects—is close to useless. Note that some archivers support searching by simply farming the work out to an external search engine such as Google [http://www.google.com/]. This is acceptable, but direct search support is usually more fine-tuned, because it allows the searcher to specify that the match must appear in a subject line versus the body, for example.

The above is just a technical checklist to help you evaluate and set up an archiver. Getting people to actually *use* the archiver to the project's advantage is discussed in later chapters, in particular the section called “Conspicuous Use of Archives”.

Software

Here are some open source tools for doing list management and archiving. If the site where you're hosting your project already has a default setup, then you may not ever have to decide on a tool at all. But if you must install one yourself, these are some possibilities. The ones I have actually used are Mailman, Ezmlm, MHonArc, and Hypermail, but that doesn't mean the others aren't good too (and of course, there are probably other tools out there that I just didn't happen to find, so don't take this as a complete list).

Mailing list management software:

- **Mailman** — http://www.list.org/

  (Has built-in archiver, and hooks for plugging in external archivers.)

- **SmartList** — http://www.procmail.org/

  (Meant to be used with the Procmail mail processing system.)

- **Ecartis** — http://www.ecartis.org/

- **ListProc** — http://listproc.sourceforge.net/

- **Ezmlm** — http://cr.yp.to/ezmlm.html

(Designed to work with the Qmail [http://cr.yp.to/qmail.html] mail delivery system.)

- **Dada** — http://mojo.skazat.com/

  (Despite the web site's bizarre attempts to hide the fact, this is free software, released under the GNU General Public License. It also has a built-in archiver.)

Mailing list archiving software:

- **MHonArc** — http://www.mhonarc.org/
- **Hypermail** — http://www.hypermail.org/
- **Lurker** — http://sourceforge.net/projects/lurker/
- **Procmail** — http://www.procmail.org/

  (Companion software to SmartList, this is a general mail processing system that can, apparently, be configured as an archiver.)

Version Control

A *version control system* (or *revision control system*) is a combination of technologies and practices for tracking and controlling changes to a project's files, in particular to source code, documentation, and web pages. If you have never used version control before, the first thing you should do is go find someone who has, and get them to join your project. These days, everyone will expect at least your project's source code to be under version control, and probably will not take the project seriously if it doesn't use version control with at least minimal competence.

The reason version control is so universal is that it helps with virtually every aspect of running a project: inter-developer communications, release management, bug management, code stability and experimental development efforts, and attribution and authorization of changes by particular developers. The version control system provides a central coordinating force among all of these areas. The core of version control is *change management*: identifying each discrete change made to the project's files, annotating each change with metadata like the change's date and author, and then replaying these facts to whoever asks, in whatever way they ask. It is a communications mechanism where a change is the basic unit of information.

This section does not discuss all aspects of using a version control system. It's so all-encompassing that it must be addressed topically throughout the book. Here, we will concentrate on choosing and setting up a version control system in a way that will foster cooperative development down the road.

Version Control Vocabulary

This book cannot teach you how to use version control if you've never used it before, but it would be impossible to discuss the subject without a few key terms. These terms are useful independently of any particular version control system: they are the basic nouns and verbs of networked collaboration, and will be used generically throughout the rest of this book. Even if there were no version control systems in the world, the problem of change management would remain, and these words give us a language for talking about that problem concisely.

"Version" Versus "Revision"

> The word *version* is sometimes used as a synonym for "revision", but I will not use it that way in this book, because it is too easily confused with "version" in the sense of a version of a piece of software—that is, the release or edition number, as in "Version 1.0". However, since the phrase "version control" is already standard, I will continue to use it as a synonym for "revision control" and "change control".

commit
: To make a change to the project; more formally, to store a change in the version control database in such a way that it can be incorporated into future releases of the project. "Commit" can be used as a verb or a noun. As a noun, it is essentially synonymous with "change". For example: "I just committed a fix for the server crash bug people have been reporting on Mac OS X. Jay, could you please review the commit and check that I'm not misusing the allocator there?"

log message
: A bit of commentary attached to each commit, describing the nature and purpose of the commit. Log messages are among the most important documents in any project: they are the bridge between the highly technical language of individual code changes and the more user-oriented language of features, bugfixes, and project progress. Later in this section, we'll look at ways to distribute log messages to the appropriate audiences; also, the section called “Codifying Tradition” in Chapter 6, *Communications* discusses ways to encourage contributors to write concise and useful log messages.

update
: To ask that others' changes (commits) be incorporated into your local copy of the project; that is, to bring your copy "up-to-date". This is a very common operation; most developers update their code several times a day, so that they know they're running roughly the same thing the other developers are running, and so that if they see a bug, they can be pretty sure it hasn't been fixed already. For example: "Hey, I noticed the indexing code is always dropping the last byte. Is this a new bug?" "Yes, but it was fixed last week—try updating, it should go away."

repository
: A database in which changes are stored. Some version control systems are centralized: there is a single, master repository, which stores all changes to the project. Others are decentralized: each developer has his own repository, and changes can be swapped back and forth between repositories arbitrarily. The version control system keeps track of dependencies between changes, and when it's time to make a release, a particular set of changes is approved for that release. The question of whether centralized or decentralized is better is one of the enduring holy wars of software development; try not to fall into the trap of arguing about it on your project lists.

checkout
: The process of obtaining a copy of the project from a repository. A checkout usually produces a directory tree called a "working copy" (see below), from which changes may be committed back to the original repository. In some decentralized version control systems, each working copy is itself a repository, and changes can be pushed out to (or pulled into) any repository that's willing to accept them.

working copy
: A developer's private directory tree containing the project's source code files, and possibly its web pages or other documents. A working copy also contains a little bit of metadata managed by the version control system, telling the working copy what repository it comes from, what "revisions" (see below) of the files are present, etc. Generally, each developer has his own working copy, in which he makes and tests changes, and from which he commits.

revision, *change*, *changeset*
: A "revision" is usually one specific incarnation of a particular file or directory. For example, if the

project starts out with revision 6 of file F, and then someone commits a change to F, this produces revision 7 of F. Some systems also use "revision", "change", or "changeset" to refer to a set of changes committed together as one conceptual unit.

These terms occasionally have distinct technical meanings in different version control systems, but the general idea is always the same: they give a way to speak precisely about exact points in time in the history of a file or a set of files (say, immediately before and after a bug is fixed). For example: "Oh yes, she fixed that in revision 10" or "She fixed that in revision 10 of foo.c."

When one talks about a file or collection of files without specifying a particular revision, it is generally assumed that one means the most recent revision(s) available.

diff
: A textual representation of a change. A diff shows which lines were changed and how, plus a few lines of surrounding context on either side. A developer who is already familiar with some code can usually read a diff against that code and understand what the change did, and even spot bugs.

tag
: A label for a particular collection of files at specified revisions. Tags are usually used to preserve interesting snapshots of the project. For example, a tag is usually made for each public release, so that one can obtain, directly from the version control system, the exact set of files/revisions comprising that release. Common tag names are things like `Release_1_0`, `Delivery_00456`, etc.

branch
: A copy of the project, under version control but isolated, so that changes made to the branch don't affect the rest of the project, and vice versa, except when changes are deliberately "merged" from one side to the other (see below). Branches are also known as "lines of development". Even when a project has no explicit branches, development is still considered to be happening on the "main branch", also known as the "main line" or "*trunk*".

  Branches offer a way to isolate different lines of development from each other. For example, a branch can be used for experimental development that would be too destabilizing for the main trunk. Or conversely, a branch can be used as a place to stabilize a new release. During the release process, regular development would continue uninterrupted in the main branch of the repository; meanwhile, on the release branch, no changes are allowed except those approved by the release managers. This way, making a release needn't interfere with ongoing development work. See the section called “Use branches to avoid bottlenecks” later in this chapter for a more detailed discussion of branching.

merge (a.k.a. port)
: To move a change from one branch to another. This includes merging from the main trunk to some other branch, or vice versa. In fact, those are the most common kinds of merges; it is rare to port a change between two non-main branches. See the section called “Singularity of information” for more about this kind of merging.

  "Merge" has a second, related meaning: it is what the version control system does when it sees that two people have changed the same file but in non-overlapping ways. Since the two changes do not interfere with each other, when one of the people updates their copy of the file (already containing their own changes), the other person's changes will be automatically merged in. This is very common, especially on projects where multiple people are hacking on the same code. When two different changes *do* overlap, the result is a "conflict"; see below.

conflict
: What happens when two people try to make different changes to the same place in the code. All version control systems automatically detect conflicts, and notify at least one of the humans involved that their changes conflict with someone else's. It is then up to that human to *resolve* the conflict, and to communicate that resolution to the version control system.

lock

A way to declare an exclusive intent to change a particular file or directory. For example, "I can't commit any changes to the web pages right now. It seems Alfred has them all locked while he fixes their background images." Not all version control systems even offer the ability to lock, and of those that do, not all require the locking feature to be used. This is because parallel, simultaneous development is the norm, and locking people out of files is (usually) contrary to this ideal.

Version control systems that require locking to make commits are said to use the *lock-modify-unlock* model. Those that do not are said to use the *copy-modify-merge* model. An excellent in-depth explanation and comparison of the two models may be found at http://svnbook.red-bean.com/svnbook-1.0/ch02s02.html. In general, the copy-modify-merge model is better for open source development, and all the version control systems discussed in this book support that model.

Choosing a Version Control System

As of this writing, the two most popular version control systems in the free software world are *Concurrent Versions System* (*CVS*, http://www.cvshome.org/) and *Subversion* (*SVN*, http://subversion.tigris.org/).

CVS has been around for a long time. Most experienced developers are already familiar with it, it does more or less what you need, and since it's been popular for a long time, you probably won't end up in any long debates about whether or not it was the right choice. CVS has some disadvantages, however. It doesn't provide an easy way to refer to multi-file changes; it doesn't allow you to rename or copy files under version control (so if you need to reorganize your code tree after starting the project, it can be a real pain); it has poor merging support; it doesn't handle large files or binary files very well; and some operations are slow when large numbers of files are involved.

None of CVS's flaws is fatal, and it is still quite popular. However, in the last few years the more recent Subversion has been gaining ground, especially in newer projects.[4]. If you're starting a new project, I recommend Subversion.

On the other hand, since I'm involved in the Subversion project, my objectivity might reasonably be questioned. And in the last few years a number of new open-source version control systems have appeared. Appendix A, *Free Version Control Systems* lists all the ones I know of, in rough order of popularity. As the list makes clear, deciding on a version control system could easily become a lifelong research project. Possibly you will be spared the decision because it will be made for you by your hosting site. But if you must choose, consult with your other developers, ask around to see what people have experience with, then pick one and run with it. Any stable, production-ready version control system will do; you don't have to worry too much about making a drastically wrong decision. If you simply can't make up your mind, then go with Subversion. It's fairly easy to learn, and is likely to remain a standard for at least a few years.

Using the Version Control System

The recommendations in this section are not targeted toward a particular version control system, and should be simple to implement in any of them. Consult your specific system's documentation for details.

Version everything

Keep not only your project's source code under version control, but also its web pages, documentation, FAQ, design notes, and anything else that people might want to edit. Keep them right next to the source code, in the same repository tree. Any piece of information worth writing down is worth versioning—that is, any piece of information that could change. Things that don't change should be archived, not versioned. For example, an email, once posted, does not change; therefore, versioning it wouldn't make sense (unless it becomes part of some larger, evolving document).

[4]See http://cia.vc/stats/vcs and http://subversion.tigris.org/svn-dav-securityspace-survey.html for evidence of this growth.

The reason versioning everything together in one place is important is so people only have to learn one mechanism for submitting changes. Often a contributor will start out making edits to the web pages or documentation, and move to small code contributions later, for example. When the project uses the same system for all kinds of submissions, people only have to learn the ropes once. Versioning everything together also means that new features can be committed together with their documentation updates, that branching the code will branch the documentation too, etc.

Don't keep *generated files* under version control. They are not truly editable data, since they are produced programmatically from other files. For example, some build systems create `configure` based on the template `configure.in`. To make a change to the `configure`, one would edit `configure.in` and then regenerate; thus, only the template `configure.in` is an "editable file." Just version the templates—if you version the result files as well, people will inevitably forget to regenerate when they commit a change to a template, and the resulting inconsistencies will cause no end of confusion.[5]

The rule that all editable data should be kept under version control has one unfortunate exception: the bug tracker. Bug databases hold plenty of editable data, but for technical reasons generally cannot store that data in the main version control system. (Some trackers have primitive versioning features of their own, however, independent of the project's main repository.)

Browsability

The project's repository should be browsable on the Web. This means not only the ability to see the latest revisions of the project's files, but to go back in time and look at earlier revisions, view the differences between revisions, read log messages for selected changes, etc.

Browsability is important because it is a lightweight portal to project data. If the repository cannot be viewed through a web browser, then someone wanting to inspect a particular file (say, to see if a certain bugfix had made it into the code) would first have to install version control client software locally, which could turn their simple query from a two-minute task into a half-hour or longer task.

Browsability also implies canonical URLs for viewing specific revisions of files, and for viewing the latest revision at any given time. This can be very useful in technical discussions or when pointing people to documentation. For example, instead of saying "For tips on debugging the server, see the www/hacking.html file in your working copy," one can say "For tips on debugging the server, see *http://svn.collab.net/repos/svn/trunk/www/hacking.html*," giving a URL that always points to the latest revision of the `hacking.html` file. The URL is better because it is completely unambiguous, and avoids the question of whether the addressee has an up-to-date working copy.

Some version control systems come with built-in repository-browsing mechanisms, while others rely on third-party tools to do it. Three such tools are *ViewCVS* (http://viewcvs.sourceforge.net/), *CVSWeb* (http://www.freebsd.org/projects/cvsweb.html), and *WebSVN* (http://websvn.tigris.org/). The first works with both CVS and Subversion, the second with CVS only, and the third with Subversion only.

Commit emails

Every commit to the repository should generate an email showing who made the change, when they made it, what files and directories changed, and how they changed. The email should go to a special mailing list devoted to commit emails, separate from the mailing lists to which humans post. Developers and other interested parties should be encouraged to subscribe to the commits list, as it is the most effective way to keep up with what's happening in the project at the code level. Aside from the obvious technical benefits of peer review (see the section called “Practice Conspicuous Code Review”), commit emails help create a sense of community, because they establish a shared environment in which people can react to events (commits) that they know are visible to others as well.

The specifics of setting up commit emails will vary depending on your version control system, but usu-

[5]For a different opinion on the question of versioning `configure` files, see Alexey Makhotkin's post "*configure.in and version control*" at http://versioncontrolblog.com/2007/01/08/configurein-and-version-control/.

ally there's a script or other packaged facility for doing it. If you're having trouble finding it, try looking for documentation on *hooks*, specifically a *post-commit hook*, also called the *loginfo hook* in CVS. Post-commit hooks are a general means of launching automated tasks in response to commits. The hook is triggered by an individual commit, is fed all the information about that commit, and is then free to use that information to do anything—for example, to send out an email.

With pre-packaged commit email systems, you may want to modify some of the default behaviors:

1. Some commit mailers don't include the actual diffs in the email, but instead provide a URL to view the change on the web using the repository browsing system. While it's good to provide the URL, so the change can be referred to later, it is also *very* important that the commit email include the diffs themselves. Reading email is already part of people's routine, so if the content of the change is visible right there in the commit email, developers will review the commit on the spot, without leaving their mail reader. If they have to click on a URL to review the change, most won't do it, because that requires a new action instead of a continuation of what they were already doing. Furthermore, if the reviewer wants to ask something about the change, it's vastly easier to hit reply-with-text and simply annotate the quoted diff than it is to visit a web page and laboriously cut-and-paste parts of the diff from web browser to email client.

   (Of course, if the diff is huge, such as when a large body of new code has been added to the repository, then it makes sense to omit the diff and offer only the URL. Most commit mailers can do this kind of limiting automatically. If yours can't, then it's still better to include diffs, and live with the occasional huge email, than to leave the diffs off entirely. Convenient reviewing and commenting is a cornerstone of cooperative development, much too important to do without.)

2. The commit emails should set their Reply-to header to the regular development list, not the commit email list. That is, when someone reviews a commit and writes a response, their response should be automatically directed toward the human development list, where technical issues are normally discussed. There are a few reasons for this. First, you want to keep all technical discussion on one list, because that's where people expect it to happen, and because that way there's only one archive to search. Second, there might be interested parties not subscribed to the commit email list. Third, the commit email list advertises itself as a service for watching commits, not for watching commits *and* occasional technical discussions. Those who subscribed to the commit email list did not sign up for anything but commit emails; sending them other material via that list would violate an implicit contract. Fourth, people often write programs that read the commit email list and process the results (for display on a web page, for example). Those programs are prepared to handle consistently-formatted commit emails, but not inconsistent human-written mails.

   Note that this advice to set Reply-to does not contradict the recommendations in the section called "The Great Reply-to Debate" earlier in this chapter. It's always okay for the *sender* of a message to set Reply-to. In this case, the sender is the version control system itself, and it sets Reply-to in order to indicate that the appropriate place for replies is the development mailing list, not the commit list.

CIA: Another Change Publication Mechanism

Commit emails are not the only way to propagate change news. Recently, another mechanism called CIA (http://cia.navi.cx/) has been developed. CIA is a real-time commit statistics aggregator and distributor. The most popular use of CIA is to send commit notifications to IRC channels, so that people logged into those channels see the commits happening in real time. Though of somewhat less technical utility than commit emails, since observers might or might not be around when a commit notice pops up in IRC, this technique is of immense *social* utility. People get the sense of being part of something alive and active, and feel that they can see progress being made right before their eyes.

The way it works is that you invoke the CIA notifier program from your post-commit hook. The

notifier formats the commit information into an XML message, and sends to a central server (typically `cia.navi.cx`). That server then distributes the commit information to other forums.

CIA can also be configured to send out RSS [http://www.xml.com/pub/a/2002/12/18/dive-into-xml.html] feeds. See the documentation at http://cia.navi.cx/ for details.

To see an example of CIA in action, point your IRC client at `irc.freenode.net`, channel `#commits`.

Use branches to avoid bottlenecks

Non-expert version control users are sometimes a bit afraid of branching and merging. This is probably a side effect of CVS's popularity: CVS's interface for branching and merging is somewhat counterintuitive, so many people have learned to avoid those operations entirely.

If you are among those people, resolve right now to conquer any fears you may have and take the time to learn how to do branching and merging. They are not difficult operations, once you get used to them, and they become increasingly important as a project acquires more developers.

Branches are valuable because they turn a scarce resource—working room in the project's code—into an abundant one. Normally, all developers work together in the same sandbox, constructing the same castle. When someone wants to add a new drawbridge, but can't convince everyone else that it would be an improvement, branching makes it possible for her to go to an isolated corner and try it out. If the effort succeeds, she can invite the other developers to examine the result. If everyone agrees that the result is good, they can tell the version control system to move ("merge") the drawbridge from the branch castle over to the main castle.

It's easy to see how this ability helps collaborative development. People need the freedom to try new things without feeling like they're interfering with others' work. Equally importantly, there are times when code needs to be isolated from the usual development churn, in order to get a bug fixed or a release stabilized (see the section called “Stabilizing a Release” and the section called “Maintaining Multiple Release Lines” in Chapter 7, *Packaging, Releasing, and Daily Development*) without worrying about tracking a moving target.

Use branches liberally, and encourage others to use them. But also make sure that a given branch is only active for exactly as long as needed. Every active branch is a slight drain on the community's attention. Even those who are not working in a branch still maintain a peripheral awareness of what's going on in it. Such awareness is desirable, of course, and commit emails should be sent out for branch commits just as for any other commit. But branches should not become a mechanism for dividing the development community. With rare exceptions, the eventual goal of most branches should be to merge their changes back into the main line and disappear.

Singularity of information

Merging has an important corollary: never commit the same change twice. That is, a given change should enter the version control system exactly once. The revision (or set of revisions) in which the change entered is its unique identifier from then on. If it needs to be applied to branches other than the one on which it entered, then it should be merged from its original entry point to those other destinations—as opposed to committing a textually identical change, which would have the same effect in the code, but would make accurate bookkeeping and release management impossible.

The practical effects of this advice differ from one version control system to another. In some systems, merges are special events, fundamentally distinct from commits, and carry their own metadata with them. In others, the results of merges are committed the same way other changes are committed, so the

primary means of distinguishing a "merge commit" from a "new change commit" is in the log message. In a merge's log message, don't repeat the log message of the original change. Instead, just indicate that this is a merge, and give the identifying revision of the original change, with at most a one-sentence summary of its effect. If someone wants to see the full log message, she should consult the original revision.

The reason it's important to avoid repeating the log message is that log messages are sometimes edited after they've been committed. If a change's log message were repeated at each merge destination, then even if someone edited the original message, she'd still leave all the repeats uncorrected—which would only cause confusion down the road.

The same principle applies to reverting a change. If a change is withdrawn from the code, then the log message for the reversion should merely state that some specific revision(s) is being reverted, *not* describe the actual code change that results from the reversion, since the semantics of the change can be derived by reading the original log message and change. Of course, the reversion's log message should also state the reason why the change is being reverted, but it should not duplicate anything from the original change's log message. If possible, go back and edit the original change's log message to point out that it was reverted.

All of the above implies that you should use a consistent syntax for referring to revisions. This is helpful not only in log messages, but in emails, the bug tracker, and elsewhere. If you're using CVS, I suggest "`path/to/file/in/project/tree:REV`", where REV is a CVS revision number such as "1.76". If you're using Subversion, the standard syntax for revision 1729 is "r1729" (file paths are not needed because Subversion uses global revision numbers). In other systems, there is usually a standard syntax for expressing the changeset name. Whatever the appropriate syntax is for your system, encourage people to use it when referring to changes. Consistent expression of change names makes project bookkeeping much easier (as we will see in Chapter 6, *Communications* and Chapter 7, *Packaging, Releasing, and Daily Development*), and since a lot of the bookkeeping will be done by volunteers, it needs to be as easy as possible.

See also the section called “Releases and Daily Development” in Chapter 7, *Packaging, Releasing, and Daily Development*.

Authorization

Most version control systems offer a feature whereby certain people can be allowed or disallowed from committing in specific sub-areas of the repository. Following the principle that when handed a hammer, people start looking around for nails, many projects use this feature with abandon, carefully granting people access to just those areas where they have been approved to commit, and making sure they can't commit anywhere else. (See the section called “Committers” in Chapter 8, *Managing Volunteers* for how projects decide who can commit where.)

There is probably little harm done by exercising such tight control, but a more relaxed policy is fine too. Some projects simply use an honor system: when a person is granted commit access, even for a sub-area of the repository, what they actually receive is a password that allows them to commit anywhere in the project. They're just asked to keep their commits in their area. Remember that there is no real risk here: in an active project, all commits are reviewed anyway. If someone commits where they're not supposed to, others will notice it and say something. If a change needs to be undone, that's simple enough—everything's under version control anyway, so just revert.

There are several advantages to the relaxed approach. First, as developers expand into other areas (which they usually will if they stay with the project), there is no administrative overhead to granting them wider privileges. Once the decision is made, the person can just start committing in the new area right away.

Second, expansion can be done in a more fine-grained manner. Generally, a committer in area X who wants to expand to area Y will start posting patches against Y and asking for review. If someone who already has commit access to area Y sees such a patch and approves of it, they can just tell the submitter

to commit the change directly (mentioning the reviewer/approver's name in the log message, of course). That way, the commit will come from the person who actually wrote the change, which is preferable from both an information management standpoint and from a crediting standpoint.

Last, and perhaps most important, using the honor system encourages an atmosphere of trust and mutual respect. Giving someone commit access to a subdomain is a statement about their technical preparedness—it says: "We see you have expertise to make commits in a certain domain, so go for it." But imposing strict authorization controls says: "Not only are we asserting a limit on your expertise, we're also a bit suspicious about your *intentions*." That's not the sort of statement you want to make if you can avoid it. Bringing someone into the project as a committer is an opportunity to initiate them into a circle of mutual trust. A good way to do that is to give them more power than they're supposed to use, then inform them that it's up to them to stay within the stated limits.

The Subversion project has operated on the honor system way for more than four years, with 33 full and 43 partial committers as of this writing. The only distinction the system actually enforces is between committers and non-committers; further subdivisions are maintained solely by humans. Yet we've never had a problem with someone deliberately committing outside their domain. Once or twice there's been an innocent misunderstanding about the extent of someone's commit privileges, but it's always been resolved quickly and amiably.

Obviously, in situations where self-policing is impractical, you must rely on hard authorization controls. But such situations are rare. Even when there are millions of lines of code and hundreds or thousands of developers, a commit to any given code module should still be reviewed by those who work on that module, and they can recognize if someone committed there who wasn't supposed to. If regular commit review *isn't* happening, then the project has bigger problems to deal with than the authorization system anyway.

In summary, don't spend too much time fiddling with the version control authorization system, unless you have a specific reason to. It usually won't bring much tangible benefit, and there are advantages to relying on human controls instead.

None of this should be taken to mean that the restrictions themselves are unimportant, of course. It would be bad for a project to encourage people to commit in areas where they're not qualified. Furthermore, in many projects, full (unrestricted) commit access has a special status: it implies voting rights on project-wide questions. This political aspect of commit access is discussed more in the section called “Who Votes?” in Chapter 4, *Social and Political Infrastructure*.

Bug Tracker

Bug tracking is a broad topic; various aspects of it are discussed throughout this book. Here I'll try to concentrate mainly on setup and technical considerations, but to get to those, we have to start with a policy question: exactly what kind of information should be kept in a bug tracker?

The term *bug tracker* is misleading. Bug tracking systems are also frequently used to track new feature requests, one-time tasks, unsolicited patches—really anything that has distinct beginning and end states, with optional transition states in between, and that accrues information over its lifetime. For this reason, bug trackers are also called *issue trackers*, *defect trackers*, *artifact trackers*, *request trackers*, *trouble ticket systems*, etc. See Appendix B, *Free Bug Trackers* for a list of software.

In this book, I'll continue to use "bug tracker" for the software that does the tracking, because that's what most people call it, but will use *issue* to refer to a single item in the bug tracker's database. This allows us to distinguish between the behavior or misbehavior that the user encountered (that is, the bug itself), and the tracker's *record* of the bug's discovery, diagnosis, and eventual resolution. Keep in mind that although most issues are about actual bugs, issues can be used to track other kinds of tasks too.

The classic issue life cycle looks like this:

1. Someone files the issue. They provide a summary, an initial description (including a reproduction recipe, if applicable; see the section called “Treat Every User as a Potential Volunteer” in Chapter 8, *Managing Volunteers* for how to encourage good bug reports), and whatever other information the tracker asks for. The person who files the issue may be totally unknown to the project—bug reports and feature requests are as likely to come from the user community as from the developers.

   Once filed, the issue is in what's called an *open* state. Because no action has been taken yet, some trackers also label it as *unverified* and/or *unstarted*. It is not assigned to anyone; or, in some systems, it is assigned to a fake user to represent the lack of real assignation. At this point, it is in a holding area: the issue has been recorded, but not yet integrated into the project's consciousness.

2. Others read the issue, add comments to it, and perhaps ask the original filer for clarification on some points.

3. The bug gets *reproduced*. This may be the most important moment in its life cycle. Although the bug is not actually fixed yet, the fact that someone besides the original filer was able to make it happen proves that it is genuine, and, no less importantly, confirms to the original filer that they've contributed to the project by reporting a real bug.

4. The bug gets *diagnosed*: its cause is identified, and if possible, the effort required to fix it is estimated. Make sure these things get recorded in the issue; if the person who diagnosed the bug suddenly has to step away from the project for a while (as can often happen with volunteer developers), someone else should be able to pick up where she left off.

   In this stage, or sometimes the previous one, a developer may "take ownership" of the issue and *assign* it to herself (the section called “Distinguish clearly between inquiry and assignment” in Chapter 8, *Managing Volunteers* examines the assignment process in more detail). The issue's *priority* may also be set at this stage. For example, if it is so severe that it should delay the next release, that fact needs to be identified early, and the tracker should have some way of noting it.

5. The issue gets scheduled for resolution. Scheduling doesn't necessarily mean naming a date by which it will be fixed. Sometimes it just means deciding which future release (not necessarily the next one) the bug should be fixed by, or deciding that it need not block any particular release. Scheduling may also be dispensed with, if the bug is quick to fix.

6. The bug gets fixed (or the task completed, or the patch applied, or whatever). The change or set of changes that fixed it should be recorded in a comment in the issue, after which the issue is *closed* and/or marked as *resolved*.

There are some common variations on this life cycle. Sometimes an issue is closed very soon after being filed, because it turns out not to be a bug at all, but rather a misunderstanding on the part of the user. As a project acquires more users, more and more such invalid issues will come in, and developers will close them with increasingly short-tempered responses. Try to guard against the latter tendency. It does no one any good, as the individual user in each case is not responsible for all the previous invalid issues; the statistical trend is visible only from the developers' point of view, not the user's. (In the section called “Pre-Filtering the Bug Tracker” later in this chapter, we'll look at techniques for reducing the number of invalid issues.) Also, if different users are experiencing the same misunderstanding over and over, it might mean that that aspect of the software needs to be redesigned. This sort of pattern is easiest to notice when there is an issue manager monitoring the bug database; see the section called “Issue Manager” in Chapter 8, *Managing Volunteers*.

Another common life cycle variation is for the issue to be closed as a *duplicate* soon after Step 1. A duplicate is when someone files an issue that's already known to the project. Duplicates are not confined to open issues: it's possible for a bug to come back after having been fixed (this is known as a *regression*), in which case the preferred course is usually to reopen the original issue and close any new reports as duplicates of the original one. The bug tracking system should keep track of this relationship bidirectionally, so that reproduction information in the duplicates is available to the original issue, and vice versa.

A third variation is for the developers to close the issue, thinking they have fixed it, only to have the original reporter reject the fix and reopen it. This is usually because the developers simply don't have access to the environment necessary to reproduce the bug, or because they didn't test the fix using the exact same reproduction recipe as the reporter.

Aside from these variations, there may be other small details of the life cycle that vary depending on the tracking software. But the basic shape is the same, and while the life cycle itself is not specific to open source software, it has implications for how open source projects use their bug trackers.

As Step 1 implies, the tracker is as much a public face of the project as the mailing lists or web pages. Anyone may file an issue, anyone may look at an issue, and anyone may browse the list of currently open issues. It follows that you never know how many people are waiting to see progress on a given issue. While the size and skill of the development community constrains the rate at which issues can be resolved, the project should at least try to acknowledge each issue the moment it appears. Even if the issue lingers for a while, a response encourages the reporter to stay involved, because she feels that a human has registered what she has done (remember that filing an issue usually involves more effort than, say, posting an email). Furthermore, once an issue is seen by a developer, it enters the project's consciousness, in the sense that that developer can be on the lookout for other instances of the issue, can talk about it with other developers, etc.

The need for timely reactions implies two things:

- The tracker must be connected to a mailing list, such that every change to an issue, including its initial filing, causes a mail to go out describing what happened. This mailing list is usually different from the regular development list, since not all developers may want to receive automated bug mails, but (just as with commit mails) the Reply-to header should be set to the development mailing list.
- The form for filing issues should capture the reporter's email address, so she can be contacted for more information. (However, it should not *require* the reporter's email address, as some people prefer to report issues anonymously. See the section called “Anonymity and involvement” later in this chapter for more on the importance of anonymity.)

Interaction with Mailing Lists

Make sure the bug tracker doesn't turn into a discussion forum. Although it is important to maintain a human presence in the bug tracker, it is not fundamentally suited to real-time discussion. Think of it rather as an archiver, a way to organize facts and references to other discussions, primarily those that take place on mailing lists.

There are two reasons to make this distinction. First, the bug tracker is more cumbersome to use than the mailing lists (or than real-time chat forums, for that matter). This is not because bug trackers have bad user interface design, it's just that their interfaces were designed for capturing and presenting discrete states, not free-flowing discussions. Second, not everyone who should be involved in discussing a given issue is necessarily watching the bug tracker. Part of good issue management (see the section called “Share Management Tasks as Well as Technical Tasks” in Chapter 8, *Managing Volunteers*) is to make sure each issue is brought to the right peoples' attention, rather than requiring every developer to monitor all issues. In the section called “No Conversations in the Bug Tracker” in Chapter 6, *Communications*, we'll look at ways to make sure people don't accidentally siphon discussions out of appropriate forums and into the bug tracker.

Some bug trackers can monitor mailing lists and automatically log all emails that are about a known issue. Typically they do this by recognizing the issue's identifying number in the subject line of the mail, as part of a special string; developers learn to include these strings in their mails to attract the tracker's notice. The bug tracker may either save the entire email, or (even better) just record a link to the mail in the regular mailing list archive. Either way, this is a very useful feature; if your tracker has it, make sure both to turn it on and to remind people to take advantage of it.

Pre-Filtering the Bug Tracker

Most issue databases eventually suffer from the same problem: a crushing load of duplicate or invalid issues filed by well-meaning but inexperienced or ill-informed users. The first step in combatting this trend is usually to put a prominent notice on the front page of the bug tracker, explaining how to tell if a bug is really a bug, how to search to see if it's already been filed, and finally, how to effectively report it if one still thinks it's a new bug.

This will reduce the noise level for a while, but as the number of users increases, the problem will eventually come back. No individual user can be blamed for it. Each one is just trying to contribute to the project's well-being, and even if their first bug report isn't helpful, you still want to encourage them to stay involved and file better issues in the future. In the meantime, though, the project needs to keep the issue database as free of junk as possible.

The two things that will do the most to prevent this problem are: making sure there are people watching the bug tracker who have enough knowledge to close issues as invalid or duplicates the moment they come in, and requiring (or strongly encouraging) users to confirm their bugs with other people before filing them in the tracker.

The first technique seems to be used universally. Even projects with huge issue databases (say, the Debian bug tracker at http://bugs.debian.org/, which contained 315,929 issues as of this writing) still arrange things so that *someone* sees each issue that comes in. It may be a different person depending on the category of the issue. For example, the Debian project is a collection of software packages, so Debian automatically routes each issue to the appropriate package maintainers. Of course, users can sometimes misidentify an issue's category, with the result that the issue is sent to the wrong person initially, who may then have to reroute it. However, the important thing is that the burden is still shared—whether the user guesses right or wrong when filing, issue watching is still distributed more or less evenly among the developers, so each issue is able to receive a timely response.

The second technique is less widespread, probably because it's harder to automate. The essential idea is that every new issue gets "buddied" into the database. When a user thinks he's found a problem, he is asked to describe it on one of the mailing lists, or in an IRC channel, and get confirmation from someone that it is indeed a bug. Bringing in that second pair of eyes early can prevent a lot of spurious reports. Sometimes the second party is able to identify that the behavior is not a bug, or is fixed in recent releases. Or she may be familiar with the symptoms from a previous issue, and can prevent a duplicate filing by pointing the user to the older issue. Often it's enough just to ask the user "Did you search the bug tracker to see if it's already been reported?" Many people simply don't think of that, yet are happy to do the search once they know someone's *expecting* them to.

The buddy system can really keep the issue database clean, but it has some disadvantages too. Many people will file solo anyway, either through not seeing, or through disregarding, the instructions to find a buddy for new issues. Thus it is still necessary for volunteers to watch the issue database. Furthermore, because most new reporters don't understand how difficult the task of maintaining the issue database is, it's not fair to chide them too harshly for ignoring the guidelines. Thus the volunteers must be vigilant, and yet exercise restraint in how they bounce unbuddied issues back to their reporters. The goal is to train each reporter to use the buddying system in the future, so that there is an ever-growing pool of people who understand the issue-filtering system. On seeing an unbuddied issue, the ideal steps are:

1. Immediately respond to the issue, politely thanking the user for filing, but pointing them to the buddying guidelines (which should, of course, be prominently posted on the web site).

2. If the issue is clearly valid and not a duplicate, approve it anyway, and start it down the normal life cycle. After all, the reporter's now been informed about buddying, so there's no point wasting the work done so far by closing a valid issue.

3. Otherwise, if the issue is not clearly valid, close it, but ask the reporter to reopen it if they get confirmation from a buddy. When they do, they should put a reference to the confirmation thread (e.g., a

URL into the mailing list archives).

Remember that although this system will improve the signal/noise ratio in the issue database over time, it will never completely stop the misfilings. The only way to prevent misfilings entirely is to close off the bug tracker to everyone but developers—a cure that is almost always worse than the disease. It's better to accept that cleaning out invalid issues will always be part of the project's routine maintenance, and to try to get as many people as possible to help.

See also the section called “Issue Manager” in Chapter 8, *Managing Volunteers*.

IRC / Real-Time Chat Systems

Many projects offer real-time chat rooms using *Internet Relay Chat* (*IRC*), forums where users and developers can ask each other questions and get instant responses. While you *can* run an IRC server from your own web site, it is generally not worth the hassle. Instead, do what everyone else does: run your IRC channels at Freenode (http://freenode.net/). Freenode gives you the control you need to administer your project's IRC channels,[6] while sparing you the not-insignificant trouble of maintaining an IRC server yourself.

The first thing to do is choose a channel name. The most obvious choice is the name of your project—if that's available at Freenode, then use it. If not, try to choose something as close to your project's name, and as easy to remember, as possible. Advertise the channel's availabity from your project's web site, so a visitor with a quick question will see it right away. For example, this appears in a prominently placed box at the top of Subversion's home page:

> *If you're using Subversion, we recommend that you join the* `users@subversion.tigris.org` *mailing list, and read the Subversion Book [http://svnbook.red-bean.com/] and FAQ [http://subversion.tigris.org/faq.html]. You can also ask questions on IRC at* `irc.freenode.net` *channel* `#svn`.

Some projects have multiple channels, one per subtopic. For example, one channel for installation problems, another for usage questions, another for development chat, etc. (the section called “Handling Growth” in Chapter 6, *Communications* discusses and how to divide into multiple channels). When your project is young, there should only be one channel, with everyone talking together. Later, as the user-to-developer ratio increases, separate channels may become necessary.

How will people know all the available channels, let alone which channel to talk in? And when they talk, how will they know what the local conventions are?

The answer is to tell them by setting the *channel topic*.[7] The channel topic is a brief message each user sees when they first enter the channel. It gives quick guidance to newcomers, and pointers to further information. For example:

```
You are now talking on #svn

Topic for #svn is Forum for Subversion user questions, see also
http://subversion.tigris.org/. || Development discussion happens in
#svn-dev. || Please don't paste long transcripts here, instead use
a pastebin site like http://pastebin.ca/. || NEWS: Subversion 1.1.0
is released, see http://svn110.notlong.com/ for details.
```

[6] There is no requirement or expectation that you donate to Freenode, but if you or your project can afford it, please consider a contribution. They are a tax-exempt charity in the U.S., and they perform a valuable service.

[7] To set a channel topic, use the `/topic` command. All commands in IRC start with "`/`". See http://www.irchelp.org/ if you're not familiar with IRC usage and administration; in particular, http://www.irchelp.org/irchelp/irctutorial.html is an excellent tutorial.

That's terse, but it tells newcomers what they need to know. It says exactly what the channel is for, gives the project home page (in case someone wanders into the channel without having first been to the project web site), mentions a related channel, and gives some guidance about pasting.

Paste Sites

An IRC channel is a shared space: everyone can see what everyone else is saying. Normally, this is a good thing, as it allows people to jump into a conversation when they think they have something to contribute, and allows spectators to learn by watching. But it becomes problematic when someone has to provide a large quantity of information at once, such as a debugging session transcript, because pasting too many lines of output into the channel will disrupt other conversations.

The solution is to use one of the *pastebin* or *pastebot* sites. When requesting a large amount of data from someone, ask them not to paste it into the channel, but instead to go to (for example) http://pastebin.ca/, paste their data into the form there, and tell the resulting new URL to the IRC channel. Anyone can then visit the URL and view the data.

There are a number of free paste sites available now, too many for a comprehensive list, but here are some of the ones I've seen used: http://www.nomorepasting.com/, http://pastebin.ca/, http://nopaste.php.cd/ http://rafb.net/paste/ http://sourcepost.sytes.net/, http://extraball.sunsite.dk/notepad.php, and http://www.pastebin.com/.

Bots

Many technically-oriented IRC channels have a non-human member, a so-called *bot*, that is capable of storing and regurgitating information in response to specific commands. Typically, the bot is addressed just like any other member of the channel, that is, the commands are delivered by "speaking to" the bot. For example:

```
<kfogel> ayita: learn diff-cmd = http://subversion.tigris.org/faq.html#diff-cmd
<ayita>  Thanks!
```

That told the bot (who is logged into the channel as ayita) to remember a certain URL as the answer to the query "diff-cmd". Now we can address ayita, asking the bot to tell another user about diff-cmd:

```
<kfogel> ayita: tell jrandom about diff-cmd
<ayita>  jrandom: http://subversion.tigris.org/faq.html#diff-cmd
```

The same thing can be accomplished via a convenient shorthand:

```
<kfogel> !a jrandom diff-cmd
<ayita>  jrandom: http://subversion.tigris.org/faq.html#diff-cmd
```

The exact command set and behaviors differ from bot to bot. The above example is with `ayita` (http://hix.nu/svn-public/alexis/trunk/), of which there is usually an instance running in `#svn` at freenode. Other bots include Dancer (http://dancer.sourceforge.net/) and Supybot (http://supybot.com/). Note that no special server privileges are required to run a bot. A bot is a client program; anyone can set one up and direct it to listen to a particular server/channel.

If your channel tends to get the same questions over and over, I highly recommend setting up a bot.

Only a small percentage of channel users will acquire the expertise needed to manipulate the bot, but those users will answer a disproportionately high percentage of questions, because the bot enables them to respond so much more efficiently.

Archiving IRC

Although it is possible to archive everything that happens in an IRC channel, it's not necessarily expected. IRC conversations may be nominally public, but many people think of them as informal, semi-private conversations. Users may be careless with grammar, and often express opinions (for example, about other software or other programmers) that they wouldn't want preserved forever in an online archive.

Of course, there will sometimes be *excerpts* that should be preserved, and that's fine. Most IRC clients can log a conversation to a file at the user's request, or failing that, one can always just cut and paste the conversation from IRC into a more permanent forum (most often the bug tracker). But indiscriminate logging may make some users uneasy. If you do archive everything, make sure you state so clearly in the channel topic, and give a URL to the archive.

RSS Feeds

RSS (Really Simple Syndication) is a mechanism for distributing meta-data-rich news summaries to "subscribers", that is, people who have indicated an interest in receiving those summaries. A given RSS source is usually called a *feed*, and the user's subscription interface is called a *feed reader* or *feed aggregator*. RSS Bandit [http://www.rssbandit.org/] and the eponymous Feedreader [http://www.feedreader.com/] are two open source RSS readers, for example.

There is not space here for a detailed technical explanation of RSS[8], but you should be aware of two main things. First, the feed reading software is chosen by the subscriber and is *the same* for all the feeds that subscriber monitors — in fact, this is the major selling point of RSS: that the subscriber chooses one interface to use for all their feeds, so each feed can concentrate just on delivering content. Second, RSS is now ubiquitous, so much so that most people who use it don't even know they're using it. To the world at large, RSS looks like a little button on a web page, with a label saying "Subscribe to this site" or "News feed". You click on the button, and from then on, your feed reader (which may well be an applet embedded in your home page) automatically updates whenever there's news from the site.

This means that your open source project should probably offer an RSS feed (note that many of the canned hosting sites — see the section called “Canned Hosting” — offer it right out of the box). Be careful not to post so many news items each day that subscribers can't separate the wheat from the chaff. If there are too many news events, people will just ignore the feed, or even unsubscribe in exasperation. Ideally, a project would offer separate feeds, one for big announcements, another following (say) events in the issue tracker, another for each mailing list, etc. In practice, this is hard to do well: it can result in interface confusion both for visitors to the project's web site and for the administrators. But at a minimum, the project should offer one RSS feed on the front page, for sending out major announcements such as releases and security alerts.[9]

Wikis

[8] See http://www.xml.com/pub/a/2002/12/18/dive-into-xml.html for that.

[9] Credit where credit is due: this section wasn't in the first published edition of the book, but Brian Aker's blog entry "Release Criteria, Open Source, Thoughts On..." [http://krow.livejournal.com/564980.html] reminded me of the usefulness of RSS feeds for open source projects.

A *wiki* is a web site that allows any visitor to edit or extend its content; the term "wiki" (from a Hawaiian word meaning "quick" or "super-fast") is also used to refer to the software that enables such editing. Wikis were invented in 1995, but their popularity has really started to take off since 2000 or 2001, boosted partly by the success of Wikipedia (http://www.wikipedia.org/), a wiki-based free-content encyclopedia. Think of a wiki as falling somewhere between IRC and web pages: wikis don't happen in real-time, so people get a chance to ponder and polish their contributions, but they are also very easy to add to, involving less interface overhead than editing a regular web page.

Wikis are not yet standard equipment for open source projects, but they probably will be soon. As they are relatively new technology, and people are still experimenting with different ways of using them, I will just offer a few words of caution here—at this stage, it's easier to analyze misuses of wikis than to analyze their successes.

If you decide to run a wiki, put a lot of effort into having a clear page organization and pleasing visual layout, so that visitors (i.e., potential editors) will instinctively know how to fit in their contributions. Equally important, post those standards on the wiki itself, so people have somewhere to go for guidance. Too often, wiki administrators fall victim to the fantasy that because hordes of visitors are individually adding high quality content to the site, the sum of all these contributions must therefore also be of high quality. That's not how web sites work. Each individual page or paragraph may be good when considered by itself, but it will not be good if embedded in a disorganized or confusing whole. Too often, wikis suffer from:

- **Lack of navigational principles.** A well-organized web site makes visitors feel like they know where they are at any time. For example, if the pages are well-designed, people can intuitively tell the difference between a "table of contents" region and a "content" region. Contributors to a wiki will respect such differences too, but only if the differences are present to begin with.
- **Duplication of information.** Wikis frequently end up with different pages saying similar things, because the individual contributors did not notice the duplications. This can be partly a consequence of the lack of navigational principles noted above, in that people may not find the duplicate content if it is not where they expect it to be.
- **Inconsistent target audience.** To some degree this problem is inevitable when there are so many authors, but it can be lessened if there are written guidelines about how to create new content. It also helps to aggressively edit new contributions at the beginning, as an example, so that the standards start to sink in.

The common solution to all these problems is the same: have editorial standards, and demonstrate them not only by posting them, but by editing pages to adhere to them. In general, wikis will amplify any failings in their original material, since contributors imitate whatever patterns they see in front of them. Don't just set up the wiki and hope everything falls into place. You must also prime it with well-written content, so people have a template to follow.

The shining example of a well-run wiki is Wikipedia, though this may be partly because the content (encyclopedia entries) is naturally well-suited to the wiki format. But if you examine Wikipedia closely, you'll see that its administrators laid a *very* thorough foundation for cooperation. There is extensive documentation on how to write new entries, how to maintain an appropriate point of view, what sorts of edits to make, what edits to avoid, a dispute resolution process for contested edits (involving several stages, including eventual arbitration), and so forth. They also have authorization controls, so that if a page is the target of repeated inappropriate edits, they can lock it down until the problem is resolved. In other words, they didn't just throw some templates onto a web site and hope for the best. Wikipedia works because its founders thought carefully about how to get thousands of strangers to tailor their writing to a common vision. While you may not need the same level of preparedness to run a wiki for a free software project, the spirit is worth emulating.

For more information about wikis, see http://en.wikipedia.org/wiki/Wiki. Also, the first wiki remains

alive and well, and contains a lot of discussion about running wikis: see http://www.c2.com/cgi/wiki?WelcomeVisitors, http://www.c2.com/cgi/wiki?WhyWikiWorks, and http://www.c2.com/cgi/wiki?WhyWikiWorksNot for various points of view.

Web Site

There is not much to say about setting up the project web site from a technical point of view: setting up a web server and writing web pages are fairly simple tasks, and most of the important things to say about layout and arrangement were covered in the previous chapter. The web site's main function is to present a clear and welcoming overview of the project, and to bind together the other tools (the version control system, bug tracker, etc.). If you don't have the expertise to set up a web server yourself, it's usually not hard to find someone who does and is willing to help out. Nonetheless, to save time and effort, people often prefer to use one of the canned hosting sites.

Canned Hosting

There are two main advantages to using a canned site. The first is server capacity and bandwidth: their servers are beefy boxes sitting on really fat pipes. No matter how successful your project gets, you're not going to run out of disk space or swamp the network connection. The second advantage is simplicity. They have already chosen a bug tracker, a version control system, a mailing list manager, an archiver, and everything else you need to run a site. They've configured the tools, and are taking care of backups for all the data stored in the tools. You don't need to make many decisions. All you have to do is fill in a form, press a button, and suddenly you've got a project web site.

These are pretty significant benefits. The disadvantage, of course, is that you must accept *their* choices and configurations, even if something different would be better for your project. Usually canned sites are adjustable within certain narrow parameters, but you will never get the fine-grained control you would have if you set up the site yourself and had full administrative access to the server.

A perfect example of this is the handling of generated files. Certain project web pages may be generated files—for example, there are systems for keeping FAQ data in an easy-to-edit master format, from which HTML, PDF, and other presentation formats can be generated. As explained in the section called “Version everything” earlier in this chapter, you wouldn't want to version the generated formats, only the master file. But when your web site is hosted on someone else's server, it may be impossible to set up a custom hook to regenerate the online HTML version of the FAQ whenever the master file is changed. The only workaround is to version the generated formats too, so that they show up on the web site.

There can be larger consequences as well. You may not have as much control over presentation as you would wish. Some of the canned hosting sites allow you to customize your web pages, but the site's default layout usually ends up showing through in various awkward ways. For example, some projects that host themselves at SourceForge have completely customized home pages, but still point developers to their "SourceForge page" for more information. The SourceForge page is what would be the project's home page, had the project not used a custom home page. The SourceForge page has links to the bug tracker, the CVS repository, downloads, etc. Unfortunately, a SourceForge page also contains a great deal of extraneous noise. The top is a banner ad, often an animated image. The left side is a vertical arrangement of links of little relevance to someone interested in the project. The right side is often another advertisement. Only the center of the page is devoted to truly project-specific material, and even that is arranged in a confusing way that often makes visitors unsure of what to click on next.

Behind every individual aspect of SourceForge's design, there is no doubt a good reason—good from SourceForge's point of view, such as the advertisements. But from an individual project's point of view, the result can be a less-than-ideal web page. I don't mean to pick on SourceForge; similar concerns apply to many of the canned hosting sites. The point is that there's a tradeoff. You get relief from the technical burdens of running a project site, but only at the price of accepting someone else's way of running it.

Only you can decide whether canned hosting is best for your project. If you choose a canned site, leave open the option of switching to your own servers later, by using a custom domain name for the project's "home address". You can forward the URL to the canned site, or have a fully customized home page at the public URL and hand users off to the canned site for sophisticated functionality. Just make sure to arrange things such that if you later decide to use a different hosting solution, the project's address doesn't need to change.

Choosing a canned hosting site

The largest and most well-known hosting site is SourceForge [http://www.sourceforge.net/]. Two other sites providing the same or similar services are savannah.gnu.org [http://savannah.gnu.org/] and BerliOS.de [http://www.berlios.de/]. A few organizations, such as the Apache Software Foundation [http://www.apache.org/] and Tigris.org [http://www.tigris.org/][10], give free hosting to open source projects that fit well with their missions and their community of existing projects.

Haggen So did a thorough evaluation of various canned hosting sites, as part of the research for his Ph.D. thesis, *Construction of an Evaluation Model for Free/Open Source Project Hosting (FOSPHost) sites*. The results are at http://www.ibiblio.org/fosphost/, and see especially the very readable comparison chart at http://www.ibiblio.org/fosphost/exhost.htm.

Anonymity and involvement

A problem that is not strictly limited to the canned sites, but is most often found there, is the abuse of user login functionality. The functionality itself is simple enough: the site allows each visitor to register herself with a username and password. From then on it keeps a profile for that user, and project administrators can assign the user certain permissions, for example, the right to commit to the repository.

This can be extremely useful, and in fact it's one of the prime advantages of canned hosting. The problem is that sometimes user login ends up being required for tasks that ought to be permitted to unregistered visitors, specifically the ability to file issues in the bug tracker, and to comment on existing issues. By requiring a logged-in username for such actions, the project raises the involvement bar for what should be quick, convenient tasks. Of course, one wants to be able to contact someone who's entered data into the issue tracker, but having a field where she can enter her email address (if she wants to) is sufficient. If a new user spots a bug and wants to report it, she'll only be annoyed at having to fill out an account creation form before she can enter the bug into the tracker. She may simply decide not to file the bug at all.

The advantages of user management generally outweigh the disadvantages. But if you can choose which actions can be done anonymously, make sure not only that *all* read-only actions are permitted to non-logged-in visitors, but also some data entry actions, especially in the bug tracker and, if you have them, wiki pages.

[10] Disclaimer: I am employed by CollabNet [http://www.collab.net/], which sponsors Tigris.org, and I use Tigris regularly.